

Project A: Classifying Reading-Level Difficulty from Text



Owen Haber, Sam Mard

CS 135 Intro To Machine Learning

Prof. Harry Bendekgey

July 20, 2026

Problem 1: Bag-of-Words Feature Representation

Section 1A

Our bag of words (BOW) pipeline converts each document into a sparse integer count vector that showcases word frequencies as they appear in the texts. We implemented our own bag of words pipeline rather than using the scikit-learn CountVectorizer. To do so, first, we tokenized each word in a custom `tokenize_text()` function. Here, text is stripped of all spaces and punctuation. We also included stopping words to remove the most commonly used words such as "the" and "or" based on what was included in our vocabulary. Each token list is then stored as a row of the training data matrix. To further reduce passage specific noise, we removed proper nouns and passage specific names by deleting words that either began, or were mostly comprised of capital letters.

Then, we counted the token frequency across the training data to form our vocabulary set from the remaining tokens, where all words that showed up at least once were considered. This resulted in 28667 words. During cross-validation, the vocabulary set was rebuilt using only the current training fold in order to avoid leakage into the validation fold. Each document was then transformed into a count-based feature vector (not binary), where each vector position represents the frequency of a vocabulary word used in that document. In order to keep feature dimensions consistent between sets, out-of-vocabulary words in the test set were ignored and not included in the feature vectors, thus, if a token was not in the final vocabulary, it did not increment any feature vectors.

Finally, we trained an `sklearn.linear_model.LogisticRegression` classifier with an inverse L2 regularization penalty `C` over a large grid of values using 5 fold cross-validation. We selected our best `C` value based on the highest mean AUROC across folds. The final model was then retrained over the entire training set using this `C` value, and evaluated on heldout data. For this model, we only considered tokens that appeared at least 4 times, resulting in a final vocabulary set of 7611 words.

Section 1B

We used 5 fold cross-validation to train and help tune a logistic regression classifier while making sure to avoid overfitting. The entire dataset was then randomly split into five folds, where of the five folds, four were used for training and one for validation. We assigned a random state value to ensure that our random fold generation was repeatable while still ensuring a general, unbiased construction of folds. Our hyperparameter search varied the inverse L2 regularization penalty `C` over a grid of 14 different values: 1e-5, 3e-5, 1e-4, 3e-4, 5e-5, `np.logspace(-3, 1, 9)`. We then found the optimal value based on the best average AUROC averaged across the five folds to provide a threshold-independent measure of binary classification quality.

For each training fold, we built a vocabulary specific to that subset, applying token processing within that training subset only to avoid leakage into the validation fold. Each validation fold was then featurized using the corresponding training vocabulary while ignoring out of vocabulary words. Once the best `C` value was picked from the cross validation results, we trained our `sklearn.linear_model.LogisticRegression` classifier model on the full dataset, only including words that appeared at least 4 times. The pipeline used off the shelf scikit-learn functions to train and score the model; our cross-validation and feature splitting was made custom.

Section 1C

We chose logistic regression for this task, as it is simple a linear classifier that works well on high-dimensional, sparse feature data, such as the BOW we implemented. Also, we were only interested in two outcomes (lower vs. upper level reading), meaning the binary output of logistic regression would give us the range of answers we needed. To control the model complexity, we focused on the inverse L2 regularization strength `C`, which provided us a good range of overfitting and underfitting. Knowing that smaller `C` values impose stronger regularization while larger `C` values allow us to fit much closer to the training data, we chose a wide range of 14 values to test across multiple orders: 1e-5, 3e-5, 1e-4, 3e-4, 5e-5, `np.logspace(-3, 1, 9)`. This range allowed us to search both ends of the spectrum while still giving us some nice intermediate values that provided insight to where the "sweet spot" might be. The model was trained separately on each fold so that the vocabulary would only be built on the

training fold to prevent leakage into the validation fold. We then computed the AUROC on each fold and computed the average AUROC per C value based on the validation data. During this process, we used a high maximum iteration limit of 5000 and a tolerance of 10^{-3} to allow convergence across all C values without early stopping. This let us reliably check each mean AUROC and subsequently determine the best C value.



This figure displays the mean training and validation AUROC as a function of the L2 regularization strength penalty C. The dark points indicate the mean scores, while the lighter points represent the AUROC for each individual fold, giving insight into the variance at each C value. Smaller C values provide stronger regularization penalties, resulting in underfitting and low training AUROC. As C increases, the validation AUROC slowly increases while the training AUROC rapidly increases until it reaches roughly 1.00 between $C = 10^{-1}$ and 10^0 ; starting at this point, overfitting occurs, as seen by the steady decrease in validation AUROC. The mean validation AUROC peaks at approximately 0.759 when $C = 0.03162$, suggesting this is the preferred hyperparameter value. However, the similar performance of neighboring C values and the variance across folds indicate that small changes in C do not drastically affect generalization, so the evidence for the exact optimal value is not entirely decisive. Still, in this instance, there is a clear peak in validation AUROC at the specific C value of 0.03162.

Section 1D

The following figure displays the confusion matrix for the heldout data from the first validation fold:

	Predicted Negative	Predicted Positive
Actual Negative	387	133
Actual Positive	88	504

$$\text{accuracy} = \frac{TP + TN}{TP + TN + FP + FN} = \frac{387 + 504}{387 + 504 + 133 + 88} = 0.801 = 80.1\%$$

$$TPR = \frac{TP}{TP + FN} = \frac{504}{504 + 88} = 0.851 = 85.1\%, \quad TNR = \frac{TN}{TN + FP} = \frac{387}{387 + 133} = 0.744 = 74.4\%$$

The confusion matrix shows the classifier’s results on a single validation fold. Overall, the model performs well, with 504 true positives and 387 true negatives. When it makes mistakes, it is more often predicting false positives (133) than false negatives (88). This suggests that the model tends to label documents as the upper reading level (positive) more often than the lower reading level (negative). However, as seen by the TPR and FPR, it is also more accurately labeling positive cases overall.

To better understand the types of mistakes our classifier makes, we filtered the heldout data by different linguistic features, specifically document word count and complexity (based on average sentence length). We then looked at the confusion matrices for these text subsets.

100 words or less

	Pred. Neg.	Pred. Pos.
Actual Neg.	152	105
Actual Pos.	42	374

Simple (avg. sentence length ≤ 30 words)

	Pred. Neg.	Pred. Pos.
Actual Neg.	333	75
Actual Pos.	78	222

More than 100 words

	Pred. Neg.	Pred. Pos.
Actual Neg.	235	28
Actual Pos.	46	130

Complex (avg. sentence length > 30 words)

	Pred. Neg.	Pred. Pos.
Actual Neg.	54	58
Actual Pos.	10	282

Comparing these results suggests that the classifier performs better on longer, more complex documents. For longer texts, it produces much less false positives (28) and results in a higher true negative count (235) compared to shorter texts, which yield 105 false positives and only 152 true negatives. This is likely because longer passages having more textual content and vocabulary diversity, which results in easier distinctions between levels. Conversely, shorter texts lack extensive vocabulary content, resulting in many lower level passages to be mistakenly classified as upper level. The same trend shows up when viewing the complexity of the passage. For more complex sentence structures, the model produces many true positives (282) with very few false negatives (10). However, it slightly overpredicts false positives (58). For less complex passages, the model makes mistakes in both directions, with 75 false positives and 78 false negatives. This indicates that a greater variance in sentence structure improves precision. Overall, the model’s accuracy improves with longer, more complex passages, suggesting that richer linguistic content provides stronger cues for determining reading level.

Section 1E

Our ultimate test set AUROC was 0.70003, which was slightly lower than our mean AUROC of 0.759 estimated from heldout performance during cross-validation. This slight difference is expected since our model must generalize to unseen data, resulting in a few trade-offs. Although we applied preprocessing filters to drop author specific data, it is likely that author overlap between folds inflated heldout performance based on patterns the model may have learned from training data. Additionally, as seen by the performance of our model with low vs. high complexity passages, differences in passage length and content between training and test sets may have caused some discrepancies. Overall, while the model provided a good estimate of heldout error, it was slightly optimistic as a result of author overlap and low data variability.

Problem 2: Open-Ended Challenge

Section 2A

For the open ended challenge we decided to transition from the count-based BOW model and use the provided BERT embeddings. Rather than word frequency, the BERT embeddings were themselves 768 dimensional feature vectors representing contextual meaning. These embeddings inherently capture semantics, syntactic relationships, and little difference in word usage that BOW cannot represent.

Before training we standardized every column of the feature matrix using the scikit-learn standard scaler, which ensured all features had mean 0 and unit variance. This was to make sure that all features were comparable to prevent unfair weighting of one feature over another. To reduce noise and improve efficiency we additionally experimented with dimensionality reduction using the scikit-learn principle component analysis. We reduced the embeddings to the top 256 dimensions, which preserved roughly 95% of the explained variance. Using PCA fit our plan of using a linear classifier because the raw BERT embeddings capture nonlinear relationships that a logistic regression wouldn't be able to recognize, so the information loss is optimal for keeping the model from getting distracted by non-linear relationships. Implementing PCA vs. the raw BERT embeddings led to a small but effective increase in test AUROC of 0.001, confirming our hypothesis that it might help.

In later experimentation we attempted to append some of the normalized numeric features available for this problem, which resulted in only a marginal improvement (< 0.001 AUROC) which was likely because they may be collinear with some of the features of the BERT embeddings. Due to this, our final pipeline consisted of only normalized BERT embeddings with a dimension reduction that was then trained on a logistic regression classifier.

Section 2B

Like the first problem we implemented 5-fold cross-validation to train and evaluate our logistic regression classifier while avoiding overfitting. However, this time, we implemented scikit-learn Group K-Fold cross validation function, which allowed us to group embeddings by author. This ensured that the same author could appear exclusively in either the training or validation fold, but never both. This prevented the model from incorrectly inflating performance by memorizing author specific patterns and allowed it to generalize better.

Within each fold, all preprocessing steps were performed strictly on the training portion. Specifically, we fit a new standard scaler on each column of the bert embeddings from the training set, and then applied the same transformation to the corresponding validation fold. PCA was implemented in the same per fold fashion, fitting it only on the train set and then applied to the validation set. This avoided any form of data leakage between training and validation folds, which better simulated the models ability to fit on unseen data.

Like the previous problem, our model was evaluated using AUROC on each validation fold, with the mean AUROC across folds deciding the best hyperparameter. Hyperparameter selection was again performed on the inverse L2 regularization strength C over a logarithmic grid, $\text{logspace}(-9, 1, 15)$. Additionally, we manually tested a few PCA reductions to find a balance between model simplicity and representation richness.

Section 2C

We again chose logistic regression for its interpretability and robustness on high-dimensional data. Like before, the single hyperparameter C controlled the strength of L2 regularization, with small C values imposing stronger penalties (reduces variance but risks underfitting) and larger values allowing a closer fit to the training data (risk of overfitting). We evaluated 15 C values spanning 10 orders of magnitude to capture both extremes. The figure below illustrates the mean training and validation AUROC across C values on a logarithmic scale, with individual folds as the light points, and the mean as dark. As C increased, mean training AUROC rapidly approached its max at around 0.85, while validation mean AUROC clearly peaked at around $C = 10^{-4}$ and then declined, indicating a clear optimum, and mild overfitting at higher complexities. The optimal setting of $C = 10^{-4}$ produced

the highest mean validation AUROC = 0.770503, which was marginally better than the results of our search in part 1. This value was selected for the final model, and retrained on the full data set using the same per-fold scaling and PCA reductions.



This figure has notably more variance between each fold, which is due to the way BERT embeddings capture nuanced stylistic and semantic information, and the per-fold PCA. Because each author is captured differently by the embeddings, and we isolate authors to strictly training or validation per fold, variance is much higher per fold than simple word counts such as those used in the previous problem. PCA further magnifies variance by keeping the 256 "most important" features, which can be very different if every fold has a unique combination of authors each with unique writing styles and content. Thus each fold has a completely different semantic subspace.

Section 2D

The following figure displays the confusion matrix for the heldout data from the second validation fold:

	Predicted Negative	Predicted Positive
Actual Negative	321	69
Actual Positive	173	550

$$\text{accuracy} = \frac{TP + TN}{TP + TN + FP + FN} = \frac{550 + 321}{1113} = 0.783 = 78.3\%$$

$$TPR = \frac{TP}{TP + FN} = \frac{550}{550 + 173} = 0.761 = 76.1\%, \quad TNR = \frac{TN}{TN + FP} = \frac{321}{321 + 69} = 0.823 = 82.3\%$$

It is worth noting that different folds had very different confusion matrices due to the reasons discussed in the previous paragraph. Some folds had slightly higher accuracy, TPR and TNR, while many had much lower. We chose the second fold because it is more representative of how the model performed across all folds. This confusion matrix is not directly comparable to those from the first problem, as we employ group K-Fold and have completely different features.

Like problem 1, we further evaluated our models performance on different subsets of the validation data based on passage length and sentence complexity.

100 words or less

	Pred. Neg.	Pred. Pos.
Actual Negative	124	58
Actual Positive	97	418

Simple (avg. sentence length ≤ 30 words)

	Pred. Neg.	Pred. Pos.
Actual Neg.	298	45
Actual Pos.	142	195

More than 100 words

	Pred. Neg.	Pred. Pos.
Actual Negative	197	11
Actual Positive	76	132

Complex (avg. sentence length > 30 words)

	Pred. Neg.	Pred. Pos.
Actual Neg.	23	24
Actual Pos.	31	355

We found that, similar to part 1, the model performs best on longer and more complex texts, where additional context allows for better classification of upper vs. lower reading levels. Short passages exhibited higher confusion rates, with 58 false positives for shorter passages vs. only 11 for longer passages, likely due to a lack of semantic information in the BERT embedding. Texts with more complexity showed a reduction false negatives, dropping from 142 to 31. These findings mirror our BOW results, indicating that textual complexity, be it length or structure, improves accuracy and reduces misclassifications for our classifier.

Section 2E

The final model got a mean holdout AUROC = 0.770503. When trained on the entire data set using the optimal $C = 10^{-4}$ and fit on the hidden test set, it achieved an AUROC of 0.73201, which is consistent with mild overfitting to training data.

This result represents a significant improvement over the Bag-Of-Words baseline from part 1. This better generalization ability can be attributed to the semantic capabilities of BERT embeddings combined with standardized fold specific preprocessing. This showed us how high dimension embeddings can yield major performance gains even with a simple classifier when proper scaling, cross-validation, and regularization are taken into account. Although cross-validated mean AUROC for BERT was only marginally higher, the test-set AUROC improved substantially, indicating the embedded representations generalize better to unseen authors and group k-fold helped simulate true unseen data.